Evaluating Classical Machine Learning for EEG-Based Attention State Classification Under Budget Hardware Constraints

CMPT 353 – Data Science Project Report

August 4, 2026

1 Introduction and Motivation

I recently joined NeuraXtension, a neuroscience club at Simon Fraser University. The club is currently exploring projects involving brain-computer interfaces (BCIs), and one idea that came up was building an attention state classifier using electroencephalogram (EEG) data. This inspired the topic of my project: I wanted to investigate how well existing classical machine learning methods perform when constrained to the hardware specifications of the NeuroPawn, a budget consumer EEG headset with only 7 channels (F3, Fz, F4, C3, Cz, C4, Pz) sampling at 125 Hz.

Most published EEG classification research uses medical-grade equipment with 32–64 electrodes at high sampling rates. The question I set out to answer is whether classical ML techniques can still distinguish between focused and unfocused mental states when limited to just 7 channels and a low sampling rate.

2 Data Acquisition

Finding suitable data proved to be one of the most difficult parts of the project. I initially searched Kaggle but found only heavily pre-processed, single-purpose datasets, and for ones that matched attention state the only one I found ended up being poorly structured and subpar in quality. I needed a competent dataset to build a realistic BCI pipeline from scratch.

I eventually found the XJTU-EEG MEMA (Multi-label EEG for Mental Attention states) dataset: 20 subjects, 12 trials each, totalling over 1,060 minutes of raw EEG recordings. However, the dataset was hosted exclusively on Baidu Pan, a Chinese cloud platform that is effectively inaccessible from North America without a local phone number. I found someone on a subreddit willing to download the data from Baidu and transfer it to me. I paid them $16 in Ethereum. The full dataset was approximately 90 GB.

3 Data Cleaning and Preprocessing

The raw MEMA data consists of tab-separated text files inside a `For_graph` directory, organized by subject. Each file contains 32 columns (one per electrode) with hundreds of thousands of rows recorded at 500 Hz.

My preprocessing pipeline (`data_processing.py`) performs four steps:

1. **Channel selection**: I discard 25 of the 32 columns and keep only the 7 channels matching the NeuroPawn's electrode positions.
2. **Downsampling**: I use `scipy.signal.decimate` to reduce the sampling rate from 500 Hz to 125 Hz, applying an anti-aliasing filter automatically.
3. **Bandpass filtering**: A 4th-order Butterworth filter removes frequencies below 0.5 Hz (motion artifacts) and above 45 Hz (electrical noise).
4. **Windowing**: The continuous signal is segmented into non-overlapping 1-second windows (125 samples each).

Labels are derived from the filenames: `_a` files correspond to "focused" (attentive) states, while `_n` and `_r` files correspond to "unfocused" (neutral/relaxed) states. Rounds 1–3 are designated as training data and round 4 as test data. The processed windows are saved as compressed `.npz` files.

4 Feature Extraction

From each 1-second window, I extract 12 features per channel using `feature_extraction.py`, yielding 84 features total (12×7 channels). These include:

- **Band powers** (5 features): Average power spectral density in the Delta (0.5–4 Hz), Theta (4–8 Hz), Alpha (8–13 Hz), Beta (13–30 Hz), and Gamma (30–45 Hz) bands, computed via Welch's method.
- **Band ratios** (3 features): Alpha/Beta, Theta/Beta, and Theta/Alpha ratios, which prior literature has linked to attention and ADHD markers.
- **Statistical features** (4 features): Mean, standard deviation, skewness, and kurtosis of the raw time-domain signal within each window.

I then apply five feature selection strategies to produce different feature subsets for comparison:

1. **None**: All 84 features retained as a baseline.
2. **ANOVA**: Features where the F-test p-value ≤ 0.05 are kept (average $\approx$33 features retained).
3. **Feature Importance (FI)**: A Random Forest is trained and features with above-average importance scores are kept (average $\approx$24 features).

4. **Linear Correlation (LCC)**: Features with above-average absolute Pearson correlation with the label are kept (average ≈27 features).

5. **PCA**: Principal Component Analysis retaining 95% of variance (average ≈39 components).

5 Experimental Design

I evaluate five classifiers from `scikit-learn`: SVM (RBF kernel, C=1.0, max_iter=2000), KNN (k=5), Random Forest (100 trees, entropy criterion), Decision Tree (entropy), and Gradient Boosting (100 estimators). All models are wrapped in a `StandardScaler` pipeline.

Each model is tested under two evaluation scenarios:

- **Subject-Dependent**: Train on a subject's rounds 1–3, test on their round 4. This simulates a calibrated, personalized device.
- **Cross-Subject**: Leave-one-subject-out—train on all data from 19 subjects, test on the held-out subject. This simulates out-of-the-box performance on a new user.

This produces a total of $20 \times 5 \times 5 \times 2 = 1{,}000$ individual experiments. To make this computationally feasible, I parallelized the outer subject loop across all CPU cores using `joblib.Parallel`, reducing wall-clock time from an estimated 2+ hours to roughly 55 minutes on a 16-thread Ryzen 7.

6 Results

The mean baseline accuracy (always predicting the majority class) is 55.1% for Subject-Dependent and 52.5% for Cross-Subject, confirming the dataset is approximately balanced.

Figure 1: Mean accuracy across 20 subjects, grouped by model, feature method, and scenario.

6.1 Cross-Subject Results

Table 1 shows the top cross-subject results. Gradient Boosting with all 84 raw features achieved the highest mean accuracy at 56.5%, followed closely by Gradient Boosting with FI-selected features (56.1%) and Random Forest with all raw features (55.5%). SVM performed the worst, barely exceeding the 52.5% baseline regardless of feature method.

Table 1: Top 5 Cross-Subject configurations by mean accuracy.

Model	Features	Accuracy	F1 Score	Avg Train Time (s)
GBoosting	None (84)	56.5%	0.537	474.5
GBoosting	FI (≈24)	56.1%	0.529	159.8
RF	None (84)	55.5%	0.534	70.5
GBoosting	ANOVA (≈33)	55.1%	0.538	193.4
GBoosting	PCA (≈39)	55.0%	0.536	203.3

6.2 Subject-Dependent Results

Performance improved substantially in the personalized scenario. The same top-performing pair—Gradient Boosting with all 84 features—reached 62.0%, and Random Forest with all features was close behind at 61.9%. SVM performed much better here (up to 60.7% with FI features), confirming that SVMs benefit from the smaller, cleaner per-subject training sets.

6.3 Key Observations

1. **Tree ensembles dominate.** Gradient Boosting and Random Forest consistently outperformed all other models in both scenarios. These models can internally discover non-linear interactions between features, which is critical for noisy EEG data.

2. **Raw features outperformed feature selection.** Contrary to what I expected, keeping all 84 features (`none`) produced the highest accuracy for the two best models. Gradient Boosting and Random Forest appear robust enough to handle the full feature space without overfitting. Feature selection methods like PCA and ANOVA actually discarded useful signal.

3. **SVM struggled on cross-subject data.** SVM with all features scored only 49.5%—below the baseline. The large, noisy cross-subject training set overwhelmed the RBF kernel, and capping `max_iter` at 2000 likely prevented convergence.

4. **Subject-dependent vs. cross-subject gap.** There is roughly a 6–7 percentage point drop from subject-dependent to cross-subject accuracy across all configurations, reflecting the high inter-subject variability of EEG signals.

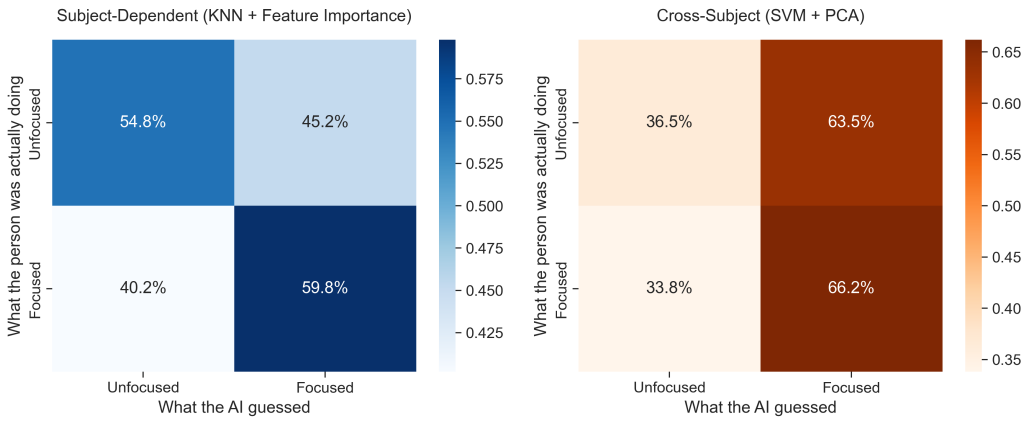

Figure 2: Aggregated confusion matrices across all 20 subjects for KNN+FI (subject-dependent) and SVM+PCA (cross-subject).

7 Limitations and Future Work

Several limitations should be noted. First, the best cross-subject accuracy of 56.5% is only modestly above the 52.5% baseline. While the improvement is consistent across subjects, it is not yet high enough for a reliable real-time BCI product. Second, capping SVM at 2000 iterations was necessary to keep training times manageable, but this likely hurt SVM's final performance. Third, 1-second windows may be too short to capture the slower brainwave dynamics associated with sustained attention; longer windows or overlapping segments could improve results.

Most importantly, this project used research-grade data that was artificially restricted to 7 channels. A real NeuroPawn headset would introduce additional noise from dry electrodes, lower signal quality, and motion artifacts. Validating these findings on actual consumer-grade hardware is a necessary next step.

If I had more time, I would explore deep learning architectures such as EEGNet or temporal convolutional networks, which can learn spatial and temporal features directly from raw waveforms rather than relying on handcrafted PSD features.

8 Conclusion

This project built an end-to-end pipeline from raw 90 GB EEG text files to trained classifiers, simulating the constraints of a budget 7-channel headset. The results show that tree-based ensemble methods—particularly Gradient Boosting—can distinguish focused from unfocused mental states above baseline, and that aggressive feature reduction is not necessarily beneficial when using models that are inherently robust to high-dimensional noisy inputs. These findings provide NeuraXtension with a concrete starting point: Gradient Boosting on the full PSD-derived feature set is the strongest classical baseline, but meaningful cross-subject accuracy will likely require either per-user calibration or more advanced architectures.

Project Experience Summary

EEG Attention State Classifier – Brain-Computer Interface

- Built an end-to-end machine learning pipeline to classify mental attention states (Focused vs. Unfocused) from raw EEG data, constrained to a 7-channel, 125 Hz consumer headset specification.
- Processed a 90 GB raw dataset (20 subjects, 1,060+ minutes): downsampled from 500 Hz, applied bandpass filtering, segmented into 1-second epochs, and extracted 84 frequency-domain and statistical features per epoch using NumPy and SciPy.
- Evaluated 5 classifiers $\times$ 5 feature selection methods $\times$ 2 evaluation scenarios across 1,000 individual experiments, identifying Gradient Boosting with full-spectrum features as the strongest classical approach (62.0% subject-dependent, 56.5% cross-subject accuracy).
- Reduced training wall-clock time by 75% by parallelizing the leave-one-subject-out evaluation loop across 16 CPU threads using Joblib.